

Take-Home Assignment

Wallet Watcher and Withdrawal Builder

Objective

Implement a service that generates a set of wallets, tracks their balances, and builds withdrawal transactions from them. The task is a self-contained slice of a custody backend.

Scope

- **Network:** implement against a single testnet of your choice — EVM (e.g. Sepolia), TRON (Shasta/Nile), or Bitcoin (testnet/signet).
- **Asset:** support a single asset — either a native coin (ETH / TRX / BTC) or one custom token (ERC-20 / TRC-20 / BRC-20).

Requirements

1. **Wallet generation.** Generate N wallets, where N is configurable with a valid range of 1 to 20 inclusive. The generation/derivation method is left to your discretion, but must be deterministic and index-ordered across runs: running the service with the same N must always produce the identical list of wallets; running with $N + m$ must produce the original N wallets plus m new ones; running with $N - m$ must produce a subset of the wallets produced for N . Equivalently, the wallet at a given index is always the same across runs.
2. **Balance tracking.** Track the balance of each wallet. A balance change on-chain must be reflected by the service within 10 minutes. Persist balances (database, file, or equivalent).
3. **Balance-change log.** Record balance changes per wallet. Inflows are detected through tracking. Outflows are recorded only when a withdrawal is actually broadcast (see Requirement 4); a transaction that is only built or simulated does not produce an outflow.
4. **Withdrawal.** Build a transaction that spends from a selected wallet to a specified destination address. Broadcasting is optional. If the transaction is not broadcast, the endpoint must return a JSON object of the built transaction, including its payload and the signature produced by the corresponding wallet. If the transaction is broadcast, send it and record the outflow.
5. **API.** Expose endpoints to: (a) retrieve all wallet addresses and their current balances; (b) create, and optionally broadcast, a withdrawal transaction from a wallet to a destination address.
6. **Tests.** Add unit tests on the wallet-generation and transaction-building functions.

Deposit handling

Do not implement deposit-transaction aggregation. Instead, describe in the README how the service behaves when multiple deposits arrive within a single polling interval (if polling is used), and when the service is offline.

Stack and tooling

The service must be implemented in **Node.js** and **TypeScript**. Any blockchain tooling is acceptable: ethers v6, TronWeb, bitcoinjs-lib, or any alternative, including constructing raw transactions and calling a node's RPC/API directly.

Constraints

- Testnet only. No mainnet, real funds, or real keys.
- Do not commit secrets.

Deliverables

- A Git repository.
- A README covering how to run the service, the deposit-handling explanation, and any trade-offs made.

Evaluation

- Correctness of wallet generation, balance tracking, and transaction construction and signing.
- Handling of amounts and precision.
- Security practices around key material and secrets.
- Code structure, tests, and documentation.